

深圳大学实验报告

课程名称 计算机视觉

项目名称 实验二：图像处理综合-路沿检测

学 院 数学科学学院

专 业 信息与计算科学（数学与计算机实验班）

指导教师 沈琳琳、 周杰

报 告 人 黄嘉聪 学号 2023192044

实验时间 2024 年 10 月 14 日至 2024 年 11 月 18 日

实验报告提交时间 2024 年 11 月 18 日

教务处制

一、实验目的与要求

实验目的：

1. 熟悉图像处理基本操作；
2. 掌握图像边缘检测原理；
3. 掌握图像基本特征抽取以及在实际问题中的应用；

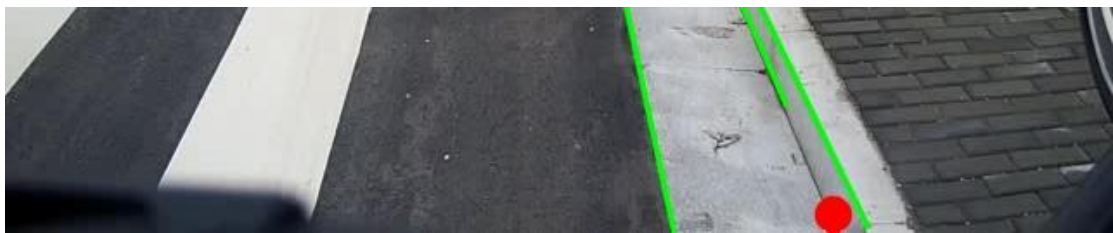
实验要求：

1. 实验提交文件为实验报告和相关程序代码，以压缩包的形式提交，实验报告命名规则为“计算机视觉-学号-姓名-实验报告 2.doc”，其他文件打包成压缩文件，命名为“计算机视觉-学号-姓名-实验报告 2-其他.zip”；
2. 所有素材和参考材料需列明出处，实验报告中的图片和程序代码建议标注个人水印或标识信息：姓名，班级，学号信息；

二、实验内容与方法

实验内容：针对给定的视频，利用图像处理基本方法实现道路路沿的检测；

提示：可利用 Hough 变换进行线检测，融合路沿的结构信息实现路沿边界定位（图中红色的点位置）。



实验方法与设计：

1. 使用 Python 的 OpenCV 库提取视频当中的每一帧，处理完成后将图片重新组成新的视频。
2. 对提取出的视频帧进行预处理，提高图像当中路沿的识别度，然后使用 Canny 算法进行边缘检测。
3. 然后使用 Hough 变换检测图像中路沿的直线，绘制检测出的直线，返回处理后的图像进行输出。
4. 对图像处理的参数进行调整，提高路沿检测效果。

三、实验步骤与过程

(1) 使用 Python 的 OpenCV 库提取视频当中的每一帧，处理完成后将图片重新组成新的视频并输出。

OpenCV 库当中有库函数 `videoCapture`、`videoWriter`，其分别用于打开视频文件，与编写与保存视频。有 `isOpened` 库函数用于逐帧读取视频。使用上述库函数与 Python 当中的读取文件函数编写代码中的视频读取与保存板块：

```

52 def process_video(input_path, output_path, frame_output_prefix):
53     # 打开视频文件
54     cap = cv2.VideoCapture(input_path)
55     # 定义视频编解码器并创建VideoWriter对象
56     fourcc = cv2.VideoWriter_fourcc(*'XVID')
57     out = cv2.VideoWriter(output_path, fourcc, 20.0, (int(cap.get(3)), int(cap.get(4))))
58
59     frame_count = 0
60     while cap.isOpened():
61         # 逐帧读取视频
62         ret, frame = cap.read()
63         if not ret:
64             break
65
66         frame_count += 1
67         # 处理每一帧
68         gray, blur, edges, = process_frame(frame)
69
70         # 如果是第十帧，保存每一步处理后的图像
71         if frame_count == 10:
72             save_intermediate_frames(gray, blur, edges, frame_output_prefix)
73
74         # 在处理后的帧上绘制直线
75         processed_frame = draw_lines(frame, edges)
76         # 将处理后的帧写入输出视频
77         out.write(processed_frame)
78
79     # 释放视频捕获和写入对象
80     cap.release()
81     out.release()
82
83 # 处理视频文件列表
84 video_files = ["E:\\nitian\\shijue\\2\\01.avi", "E:\\nitian\\shijue\\2\\02.avi", "E:\\nitian\\shijue\\2\\03.avi"]
85 for video_file in video_files:
86     # 为每个输入视频生成输出文件路径
87     output_file = video_file.replace(".avi", "_processed.avi")
88     # 为每个输入视频生成第十帧的输出图像前缀
89     frame_output_prefix = video_file.replace(".avi", "_frame10")
90     # 处理视频并保存结果
91     process_video(video_file, output_file, frame_output_prefix)

```

图一 视频提取与合成模块

其中的处理函数在后续步骤中实现。

(2) 对提取出的视频帧进行预处理，提高图像当中路沿的识别度，以便后续算法进行识别。

该环节中，通过在 process_frame 函数当中对于图像进行了灰度化与高斯模糊的预处理，显著提升了图像当中路沿的识别度：

① 灰度化

提取的图像为彩色图像，使用 RGB 模式对图像数据进行存储，干扰信息过多，无法运用边缘检测函数，故对图像进行灰度化，使之只剩余亮度信息，简化图像内容，减少多余的信息所带来的噪声干扰

例图：



图二 视频一第十帧的原始图像

使用 OpenCV 中的 `cvtColor` 函数对图像进行灰度化处理，代码如下：

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

处理过后的例图如下：

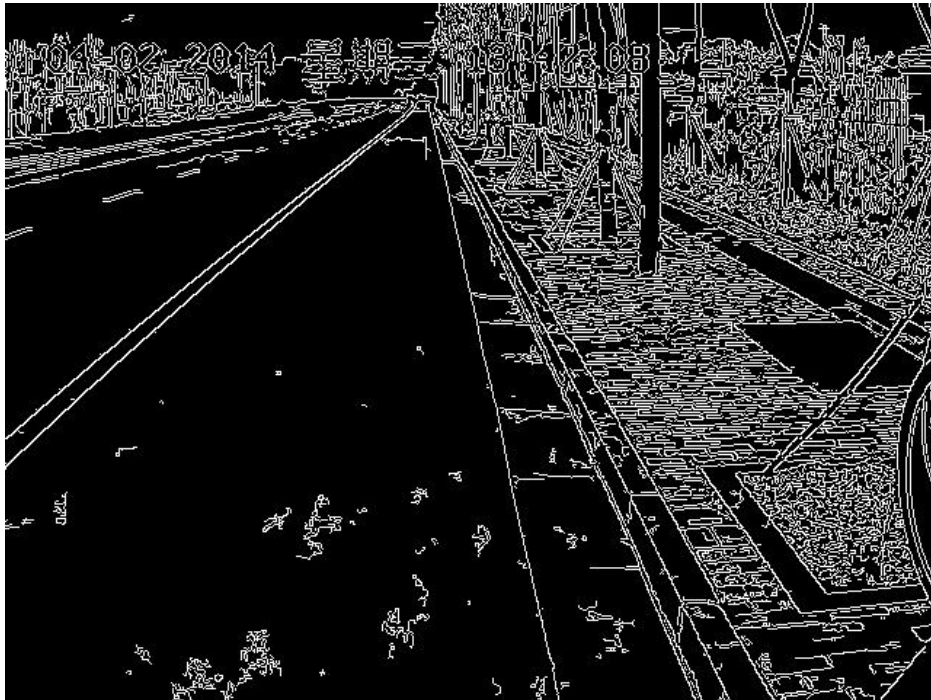


图三 灰度化处理后的原始图像

在处理过后，图像当中只剩余亮度信息，图像的内容被极大简化。

② 高斯模糊

直接对上述灰度化的图像进行 Canny 算法检测边缘后，发现图像中由于右侧路径当中存在许多较小的边缘，导致检测完边缘后的图像当中存在非常明显的噪声，需要对噪声进行消除



图四 直接对灰度化后的图像进行 Canny 边缘检测后图像

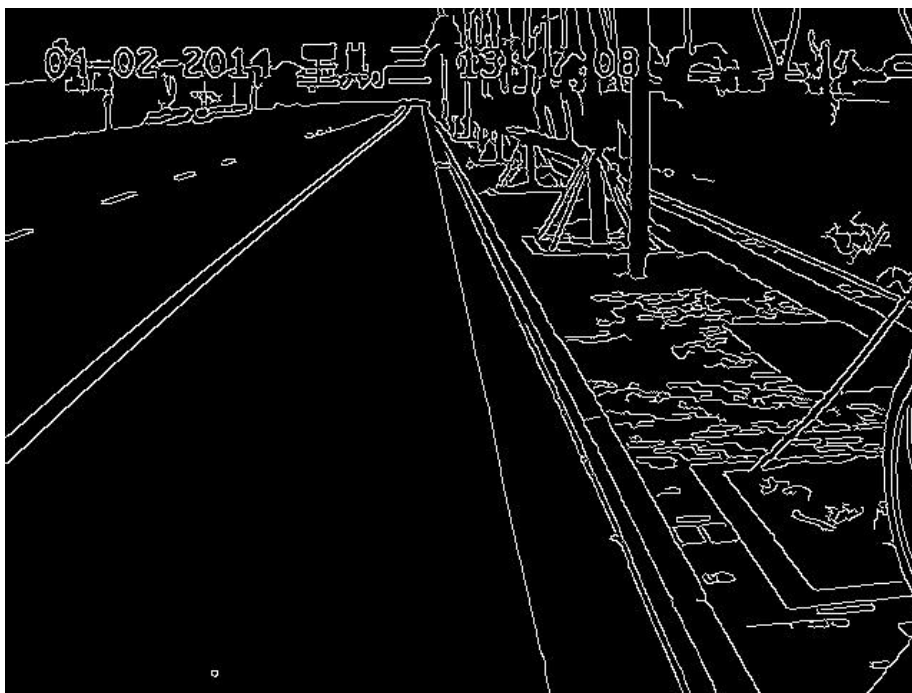
高斯滤波器可以对图像进行简易的模糊，可以显著的去掉图像中的噪声使用 OpenCV 中的 `cvtColor` 函数对图像进行灰度化处理，代码如下：

```
blur = cv2.GaussianBlur(gray, (7, 7), 0)
```



图五 对灰度化后的图像进行高斯模糊后的图像

此时，对该图像再次使用 Canny 算法，产生图像如下：



图六 高斯模糊后进行 Canny 边缘检测后图像

对比图四，可以看到高斯模糊过后，消除了大量的噪点信息，同时路沿的信息也被完整的保存，以便后续识别

(3) 然后使用 Hough 变换检测图像中路沿的直线，绘制检测出的直线，返回处理后的图像进行输出。

该环节中，通过在 draw_lines 函数当中对于图像进行了 Hough 变换，并返回处理完成的图像：

可以发现在图六的所有直线当中，只有路沿处的直线斜率较大，其他的直线斜率过小或者说斜率为负数，故使用提供一组参数对 (θ, r_0) 的标准霍夫线变换 OpenCV 当中实现标准霍夫线变换的是 HoughLines 库函数。使用该函数对图像进行 Hough 变换：

```
lines = cv2.HoughLines(edges, 1, np.pi / 180, 220)
```

然后遍历检测出的每一条线，对 theta 进行筛选，剩余路沿所对应的直线

```
if theta < 2:  
    continue
```

使用 np 库对计算直线的位置，然后进行绘制，最后返回处理好的图像：

```
# 计算cos(theta)和sin(theta)
a = np.cos(theta)
b = np.sin(theta)
# 计算rho对应的x和y坐标
x0 = a * rho
y0 = b * rho
# 计算x1和y1的坐标
x1 = int(x0 - 350 * (-b))
y1 = int(y0 - 350 * a)
# 计算x2和y2的坐标
x2 = int(x0 - 700 * (-b))
y2 = int(y0 - 700 * a)
# 在图像上绘制直线
cv2.line(frame, (x1, y1), (x2, y2), (0, 255, 0), 4)

return frame
```

图七 在图像上绘制直线并返回图像

最后对代码当中的参数进行调整，获取最佳的输出结果。最后确定以 7*7 变换的单次高斯模糊等参数进行输出为最佳情况。

输出结果：



图八 视频三最终输出结果某一帧截图

四、实验结论或体会

实验结论：

经过本次实验，我深入了解了图像处理的基本操作，掌握了图像边缘检测的原理以及图像基本特征的抽取方法，并将其成功应用于实际问题——道路路沿的检测中。

在实验过程中，首先通过 Python 的 OpenCV 库提取了视频中的每一帧，并对这些帧进行了预处理，包括灰度化和高斯模糊。灰度化处理简化了图像内容，只保留了亮度信息，从而减少了多余信息带来的噪声干扰。而高斯模糊则进一步消除了图像中的噪声，使得后续的边缘检测更加准确。

在边缘检测阶段，采用了 Canny 算法。经过高斯模糊处理后的图像，在使用 Canny 算法进行边缘检测时，噪声明显减少，路沿的边缘也更加清晰。

接下来，我使用了 Hough 变换来检测图像中的直线。通过设定合适的参数，我成功地筛选出了路沿所对应的直线。这一过程中，我深刻体会到了 Hough 变换在直线检测中的强大功能，它能够有效地从复杂的图像中提取出我们感兴趣的直线特征。

在实验的最后阶段，我将检测出的路沿直线绘制在原始图像上，并将处理后的图像重新组合成视频进行输出。通过调整图像处理的参数，最后输出了效果最佳的视频。通过对比原始视频和处理后的视频，可以清晰地看到了路沿检测的效果，这进一步验证了实验方法和步骤的正确性。

实验体会：

在本次实验的过程当中使用了许多方法-二值化、图像均衡化等去除图像的噪声。但由于在二值化过后图像被抹去了大量数据，导致路沿的部分信息丢失，无法进行识别，识别效果差，故在代码中删除了相关的部分。图像均衡化后使得图像的对比度加强，导致图像右侧的人行道上的线条的差距被显著加强，从而产生了更多不正常的线路，故在代码中也删去了相关的部分。

在编写的过程当中，对于角度的限制虽然对于给予的视频效果很好，但是对于更多种类视频泛化性不够，如在香港等地区识别路沿时，其路沿会在视频左侧，此时实验当中的方法适用性就不足了。

最后，通过这一次实验，我深刻的学到了处理视频、识别路沿的方法等，深刻的认识到了 Hough 变换及辅助其实现的方法的作用与其应用，也学习到了各个参数对于图像处理的作用。

指导教师批阅意见：
成绩评定：
指导教师签字：沈琳琳、周杰
2024 年 11 月 18 日
备注：

注： 1、报告内的项目或内容设置，可根据实际情况加以调整和补充。